Assignment 1 Report

Approach

This assignment was about incrementally building a **high-performance, causal FlashAttention stack**. I started with the forward pass (Problems 1–7), and then extended into training-ready kernels with backward passes (Problems 8–9).

My overall approach was centered on:

- **Numerically stable streaming softmax** (maintaining per-row running stats instead of forming the full attention matrix).
- **Efficient memory access** using Triton (careful tiling, pointer arithmetic, coalesced loads).
- **Algorithmic constraints** for causal masking, grouped-query attention (GQA), sliding windows, and attention sinks.

I Learnt a lot from the assignment, especially from problems 6-9.

Problem 6: Sliding Window Attention (SWA)

I implemented a causal FlashAttention forward kernel that restricts each query token to attend only within a **fixed recent window**.

Key ideas:

- Computed **per-block query offsets** and determined `window_start` to bound the key/value traversal.
- Used an **online softmax in the base-2 domain** to maintain numerical stability:
    - Running max `m_i` per row.
    - Running denominator `l_i` per row.
- **Performance tricks**:
    - Tiling with `BLOCK_M = 128` and `BLOCK_N = 64`.
    - Head mapping (for GQA-enabled code) and stride-aware pointer arithmetic for coalesced loads.
    - Two-phase traversal: handle **off-diagonal tiles** first, then **diagonal tiles** with in-tile causal masking.

Problem 7: SWA + Attention Sinks

I extended the forward kernel by adding **attention sinks**:

- **Phase 0:** Attend to early "sink" tokens (`indices < sink_size`).
- **Phase 1:** Attend to off-diagonal tiles within the sliding window.
- **Phase 2:** Attend to diagonal tiles with causal, window, and sink masks applied.

The valid mask combined:

> Causal constraint， Window constrain, Sink constraint， Bounds constraint

Problem 8: GQA Causal FlashAttention with Backward Pass

This was the big one, implementing both forward and backward for **Grouped Query Attention (GQA)**.

Forward (GQA)

- Mapped each query head to a shared K/V head:
  kv_head_idx = q_head_idx // (N_Q_HEADS // N_KV_HEADS)
- Checked divisibility to ensure exact grouping.
- Reused the **streaming softmax** with per-row `m_i` and `l_i`, saved during forward for backward reconstruction.

Backward (core logic)

- Reconstructed probabilities using saved stats (M and L):
  p_tilde = exp2(S2 - m_i)
  P = p_tilde / l_i
- Computed gradients with the softmax-Jacobian trick:
  - `dV += P^T @ dO`
  - `delta = sum(dO ⊙ O)` per row
  - `dS = P ⊙ (dO @ V^T - delta)`
  - `dQ += dS @ K × scale`
  - `dK += dS^T @ Q × scale`
- Used **atomic adds** for `dK` and `dV` since multiple queries contribute to the same rows.

Implementation

- **Triton kernel:** streams across key tiles, accumulates `dQ` locally, uses atomic adds for `dK/dV`.

- **Python reference version:** expands K/V for GQA alignment, loops in blocks, then reduces back to KV heads.

Key insights

- Saving just `m_i` and `l_i` is enough to reconstruct probabilities **exactly**.
- Atomic accumulation is unavoidable for shared K/V.
- Always cast to **fp32** for reductions.

Problem 9: Backward for SWA + Sinks + GQA

Here I extended the backward pass to include **sliding windows and sinks**.

- Defined `window_start` and `sink_end`.
- Split traversal into 3 phases:
  - **Phase 0:** sinks before window.
  - **Phase 1:** off-diagonal tiles within window.
  - **Phase 2:** diagonal tiles with all masks applied.
- Applied the same gradient formulas as in Problem 8, but gated by the combined mask: causal $\wedge$ (within_window $\vee$ in_sink) $\wedge$ in_range

Crucially, this kept the **streaming properties**:

- No full attention matrices.
- Only reused saved `m_i` and `l_i`.
- Kept tile sizes and memory-efficient traversal.

Key Learnings

1. Streaming Softmax

- The **numerically stable online softmax** is the backbone.
- `m_i` and `l_i` let you compute exact softmax across disjoint tiles.
- Base-2 domain with `exp2` + `LOG2E` scaling avoids instability.

2. Gradient Calculations

- Core formulas learned and implemented:
  - `dV = P^T @ dO`
  - `dS = P ∘ (dO @ V^T - delta)`
  - `dQ = dS @ K × scale`
  - `dK = dS^T @ Q × scale`

- Atomic adds are essential for correctness.

3. GQA in Practice

- Learned how query-head grouping works in practice.
- Memory savings and throughput gains come from shared K/V.
- Backward pass needs careful accumulation strategies.

4. Streaming LLM Techniques

- **Sliding Window Attention** reduces compute/memory.
- **Attention Sinks** stabilize long-context streaming.
- Combining constraints (causal, window, sink, bounds) was tricky but manageable.

5. Triton Kernel Craftsmanship

- Block tiling + pointer arithmetic = performance.
- Always cast to **fp32** for reductions.
- Two-phase traversal simplifies causal masking.

6. Integration with Autograd

- Wrapped kernels in `torch.autograd.Function`.
- Saved **minimal sufficient state** (`m_i`, `l_i`).
- Balanced recompute vs save, leaned toward minimal saves.

Final Thoughts

This assignment wasn't just about implementing FlashAttention forward kernels, it was about **mastering backward passes in a streaming, numerically stable way**. Problems 8 and 9, in particular, solidified my understanding of how attention gradients are derived and efficiently implemented. Problems 6 and 7 gave me practical exposure to real-world LLM tricks like sliding windows and attention sinks.

Overall, I now feel much more confident about both the math and the low-level implementation details of modern attention mechanisms.